



دانشکده‌گان علوم و فناوری‌های میان‌رشته‌ای

دانشکده سامانه‌های هوشمند

گروه علوم و فناوری داده

درس کاربرد سیستم‌های هوشمند در پزشکی

استاد: دکتر مهدی تیموری

گردآورنده: مهدیس باقری دورفرد

INDEX

Imports.....	3
Approches to store states.....	3
Approach 1 : Matrix representation.....	3
Approach 2 : Array representation.....	4
Approach 3 : String representation.....	4
Approach 4 : Number representation	5
Answer to qestion A.....	6
Required functions for A*	7
A function for generating children	8
Heuristic Functions	9
A function for printing states	11
A function for turning state to tuple	11
A function for reconstructing path.....	11
A function for detecting move and swap	13
A* function.....	14
Report and Call	17
Analysis and Conclusion.....	19
Answer to an important question about time complexcity	20

Imports

در این بخش، ابزارهای مورد نیاز `import` شدند.

```
from copy import deepcopy
import heapq
```

استفاده از `deepcopy` برای جلوگیری از تغییر ناخواسته‌ی حالت‌های والد در هنگام تولید `successor` ها ضروری است، به‌ویژه زمانی که حالت‌ها با ساختارهای `mutable` مانند لیست‌ها نمایش داده می‌شوند.

برای پیاده‌سازی لیست `open` در الگوریتم A^* ، استفاده از ساختار داده‌ی `heap` ضروری است، زیرا می‌تواند با پیاده‌سازی صف اولویت، امکان انتخاب سریع حالتی با کمترین مقدار $f(n)$ را با پیچیدگی زمانی لگاریتمی فراهم کند.

Approches to store states

در این بخش، تلاش بر این بود که روش‌های ذخیره‌سازی حالت‌ها از نظر بهینگی و خوانایی بررسی شود.

پیدا کردن یک مقدار مشخص در همه‌ی روش‌ها در مرتبه $O(9)$ انجام می‌شود.

Approach 1 : Matrix representation

```
state = [[1, 2, 3],
         [4, 0, 5],
         [6, 7, 8]]
```

مزایا

- خواناترین بیان ممکن و فهم بصری بالا
- دسترسی به موقعیت ردیف‌ها و ستون به صورت مستقیم و ساده
- مناسب برای نمایش گرافیکی یا چاپ مرحله‌به‌مرحله و بنابراین خطایابی آسان

معایب

- مصرف حافظه بیشتر از `string` یا عدد
- قابلیت `hash` شدن پیش‌فرض ندارد، بنابراین برای قرار گرفتن در `closed set` باید تبدیل شود
- سرعت نسبی پایین‌تر نسبت به `string` و `integer`
- برای استفاده در A^* نیاز به `immutable` شدن دارد (باید `tuple of tuples` شود)

```
state = [1, 2, 3,
         4, 0, 5,
         6, 7, 8]
```

Approach 2 : List representation

مزایا

- نسبت به ماتریس کم حجم تر
- عملیات swap راحت
- ساخت successor ها نسبتاً سریع
- ساختن tuple برای hash کردن ساده تر

معایب

- باز هم mutable است
- نیاز به تبدیل به tuple برای پیاده سازی A^* دارد
- نسبت به ماتریس، خیلی خوانا نیست.
- تبدیل ردیف و ستون با divmod انجام می شود و از ماتریس سخت تر است.

Approach 3 : String representation

```
state = "123405678"
```

مزایا

- کاملاً immutable
- مناسب ترین ساختار برای قرار گرفتن در مجموعه های visited/closed
- بسیار سبک از نظر حافظه
- قابل hash شدن مستقیم (بدون تبدیل)
- سریع در مقایسه کردن دو state

معایب

- هر بار جابه جایی نیاز به ساخت string جدید دارد
- کار با رشته برای بسیاری از عملیات شهودی نیست (نسبت به ماتریس)
- تبدیل ردیف و ستون با divmod انجام می شود و از ماتریس سخت تر است.

Approach 4 : Number representation

```
state = 123405678
```

مزایا

- کمترین مصرف حافظه
- قابل مقایسه سریع

معایب

- برای جابه‌جایی باید به string تبدیل شود
- خوانایی بسیار پایین
- تبدیل زیاد بین string و int ممکن است overhead ایجاد کند
- احتمال خطا در فهم حالت نسبت به بقیه بیشتر

مقایسه نهایی

String : بهترین گزینه ترکیبی سرعت + حافظه + سادگی و Matrix : فهم آسان، ولی کند و حجیم

با این حال، به دلایل زیر، در این تمرین از نمایش ماتریسی استفاده شده است.

- خواناترین نمایش برای گزارش
- فهم حرکت‌ها و مجاورت (up/down/left/right) خیلی طبیعی‌تر است
- امکان چاپ مرحله به مرحله بسیار شهودی‌تر است.
- ساخت successor ها در ماتریس مفهوم‌تر است.
- هزینه زمانی ماتریس برای این تمرین قابل قبول است

Answer to question A

الف) در مسئله هشت پازل حالت ها، عمل ها و آزمون هدف را به طور دقیق تعریف کنید)

تعریف حالت ها (States)

هر حالت یک چیدمان 3×3 از کاشی ها شامل اعداد ۱ تا ۸ و یک خانه ی خالی (۰ یا blank) است. ویژگی ها:

- حالت ها معمولاً به صورت ماتریس 3×3 ، لیست ۹ تایی یا tuple نمایش داده می شوند.
- تنها شرط معتبر بودن یک حالت این است که:
 - هر یک از کاشی های ۱ تا ۸ دقیقاً یک بار،
 - و کاشی خالی دقیقاً یک بار باشد.

بنابراین $9! = 362880$ حالت وجود دارد.

نکته ی خیلی مهم این است که به دلیل ویژگی های جایگشتی و زوجیت (parity) در جابه جایی ها، تنها نصف این حالت ها از یکدیگر قابل رسیدن اند. این موضوع یک ویژگی بنیادی در پازل های کشویی است. بنابراین:

- تعداد کل حالت های ممکن: ۳۶۲۸۸۰
- تعداد حالت های قابل حل ۱۸۱۴۴۰
- تعداد حالت های غیر قابل حل: ۱۸۱۴۴۰

تعریف عمل ها (Actions)

در مسئله هشت پازل، تنها عمل قابل انجام جابه جایی خانه ی خالی با یکی از همسایگانش است. اگر خانه خالی در یکی از ۴ گوشه باشد، ۲ عمل مجاز وجود دارد. اگر در مرکز باشد، ۴ عمل مجاز و اگر در سایر موقعیت ها باشد، ۳ عمل مجاز وجود دارد.

به صورت رسمی اگر موقعیت خانه ی خالی را (i, j) بنامیم، عمل های مجاز عبارتند از جابه جایی (i, j) با

Up: $i-1, j$ down: $i+1, j$ left = $i, j-1$ right = $i, j+1$

البته اگر از نظر موقعیتی مجاز باشند. یعنی حاصل جمع و تفریق، صفر، یک یا دو شوند. اگر از این محدوده کمتر یا بیشتر شوند، آن عمل مجاز نیست.

تعریف آزمون هدف (Goal Test)

```
if current_t == goal_t:
```

یک حالت زمانی هدف (Goal State) محسوب می شود که کاشی ها دقیقاً در ترتیب صحیح نهایی قرار گرفته باشند.

Required functions for A*

A function for generating children

این تابع یکی از حیاتی‌ترین اجزای الگوریتم A* در مسئله هشت پازل است. A* برای هر حالت فعلی باید بتواند تمام حالت‌های قابل دستیابی با یک حرکت معتبر را تولید کند. این مجموعه حالت‌های جدید همان فرزندان (successors) هستند.

- ورودی: یک ماتریس ۳×۳ که نمایش دهنده‌ی وضعیت فعلی پازل است
 - خروجی: لیستی از حالت‌های جدیدی که با یک حرکت از حالت فعلی به وجود می‌آیند
- ابتدا موقعیت خانه خالی (۰) در ماتریس پیدا می‌شود (شماره ردیف در zr و شماره ستون در zc قرار می‌گیرد)

```
zr = zc = None
for r in range(3):
    for c in range(3):
        if state[r][c] == 0:
            zr, zc = r, c
            break
if zr is not None:
    break
```

چهار حرکت اصلی (بالا، پایین، چپ و راست) بدین صورت تعریف شده اند:

```
moves = [(-1, 0), # up
          (1, 0),  # down
          (0, -1), # left
          (0, 1)] # right
```

در حلقه for، باید هر حرکت (هر همسایه) بررسی شود. یعنی به ازای هر حرکت (بالا، پایین، چپ، راست)، یک بار این حلقه ی for اجرا میشود. (خانه خالی اگر در یکی از ۴ گوشه باشد دو همسایه، اگر مرکز باشد ۴ همسایه و در سایر موقعیت ها ۳ همسایه دارد) نحوه بررسی مجاور بودن بدین صورت است که: اندیس های موقعیت صفر با اندیس های حرکت مورد بررسی جمع میشوند و درون متغیرهای nr و nc میریزند. اگر شرط if برایشان True شود (اگر $0 \leq nr < 3$ and $0 \leq nc < 3$) از state فعلی با استفاده از کپی عمیق (deepcopy) کپی به نام new_state گرفته میشود. در new_state، موقعیت قرارگیری ۰ با موقعیت همسایه مجاور عوض میشود (swap میشوند) و در نهایت به لیست فرزندان append میشود. سپس دوباره به ابتدای حلقه ی for برگشته میشود تا حرکت های ممکن دیگر بررسی شوند)

```
for dr, dc in moves:
    nr, nc = zr + dr, zc + dc
    if 0 <= nr < 3 and 0 <= nc < 3: # check bounds
        new_state = deepcopy(state)
        # swap blank with neighbor
        new_state[zr][zc], new_state[nr][nc] = new_state[nr][nc],
        new_state[zr][zc]
        children.append(new_state)
```

Manhattan Distance function: فاصله منتهن یک هیوریستیک *admissible* و *consistent* است و تضمین می‌کند که الگوریتم A^* کم‌هزینه‌ترین مسیر را بیابد. این تابع، حالت هدف و حالت فعلی را دریافت میکند و جمع فواصل منتهن را *return* میکند.

```
def manhattan_distance(state, goal):
```

در این روش برای هر کاشی شماره‌دار، فاصله افقی و عمودی آن تا موقعیت صحیحش در حالت هدف محاسبه و جمع می‌شود.

```
goal_pos = {}
for r in range(3):
    for c in range(3):
        val = goal[r][c]
        goal_pos[val] = (r, c)
```

برای افزایش سرعت دسترسی، ابتدا یک جدول *lookup* (یک دیکشنری به نام *goal_pos*) از موقعیت تمام کاشی‌ها در حالت هدف ساخته می‌شود (تا مکان صحیح هر عدد با پیچیدگی زمانی $O(1)$ قابل دسترسی باشد).

```
for r in range(3):
    for c in range(3):
        val = state[r][c]
        if val == 0:
            continue # skip blank
        gr, gc = goal_pos[val]
        total += abs(r - gr) + abs(c - gc)
```

سپس به ازای تک تک مقادیر درایه‌های *state* داده شده، (به جز وقتی درایه صفر یا همان خانه خالی است)، موقعیت هاشان با موقعیت‌های قرارگیری همان مقدار در حالت هدف مقایسه می‌شود (فاصله عمودی و افقی شان محاسبه می‌شود) و اختلاف هاشان به متغیر *total* که در ابتدا صفر تعریف شده بود، اضافه می‌شود.

Heuristic Manhattan Distance + 2 Linear Conflict

این heuristic نسخه بهبود یافته منهتن است و مانند آن، **admissible** و **consistent** است و از آن دقیق تر نیز هست.
تابع `manhattan_conflict` حالت هدف و حالت فعلی را دریافت میکند و جمع فواصل منتهن + دو برابر تعداد `conflict` ها را `return` میکند.

```
manhattan = manhattan_distance(state, goal)
```

بنابراین ابتدا تابع منهتن را کال میکند و خروجی اش را در متغیر `Manhattan` ذخیره میکند. سپس درست مانند تابع منهتن، یک جدول `lookup` میسازد.

```
goal_pos = {}
```

```
....
```

سپس، تضاد های خطی باید شمرده شوند. ابتدا تضاد های ردیفی و بعد تضاد های ستونی شمرده میشوند و به متغیر `conflicts` افزوده میشوند. نکته خیلی مهم این است که: کاشی خالی (مقدار صفر) نباید در حلقه شمارش تضاد ها در نظر گرفته شود. حال تضاد های خطی چطور شمرده میشوند؟

```
# Row conflicts
for row in range(3):
    row_tiles = []
    for col in range(3):
        tile = state[row][col]
        if tile != 0:
            gr, gc = goal_pos[tile]
            if gr == row: # tile belongs in this row
                row_tiles.append((col, gc))
```

به ازای هر ردیف، اولین حلقه یک بار اجرا میشود. در ابتدا متغیر `row_tiles` یک لیست خالی است. سپس به ازای هر مورد (ستون) در آن ردیف، وارد حلقه دوم میشود. در این حلقه، در ابتدا مقدار یک کاشی در `tile` قرار میگیرد و (اگر آن مقدار ۰ نباشد) تابع `if` اول میشود و اندیس های همان مقدار در (حالت هدف) در متغیر های `gr` و `gc` میگیرد. در `if` دوم، بررسی میشود که آیا کاشی مورد نظر ما در حالت فعلی و حالت هدف در یک ردیف قرار دارند یا خیر. اگر شرط `True` باشد، موقعیت فعلی کاشی در ردیف به همراه موقعیت درست کاشی در ردیف هدف به صورت یک تاپل دوتایی در لیست `row_tiles`، `append` میشوند.

```
# count reversed pairs
for i in range(len(row_tiles)):
    for j in range(i + 1, len(row_tiles)):
        col1, gcol1 = row_tiles[i]
        col2, gcol2 = row_tiles[j]
        if gcol1 > gcol2:
            conflicts += 1
```

حلقه ی for اول: در این بخش، به ازای هر زوج ذخیره شده در لیست row_tiles، وارد حلقه for اول میشود. یکی از مهم ترین خط های این قطعه کد، حلقه for دومی است.

به ازای هر j در محدوده (i + 1, len(row_tiles))، وارد این حلقه میشویم. علت شروع مقدار دهی از i+1 این است که یک مقدار با خودش مقایسه نشود.

برای مثال اگر row_tiles ۳ عضو داشته باشد:

وقتی i = 0، j مقادیر ۱ و ۲ میگیرد. پس 0 یکبار با ۱ و یکبار با ۲ مقایسه میشود.

وقتی i = 1، j مقدار ۲ میگیرد. پس یکبار با ۲ مقایسه میشود. (از مقایسه تکراری پرهیز میشود)

سپس موقعیت (اندیس ستونی) مقادیر j, i در حالت هدف، در متغیر های gcol2 و gcol1 قرار میگیرد.

```
if gcol1 > gcol2:
    conflicts += 1
```

در واقع اندیس ستون مقدار i در حالت مورد بررسی کمتر از اندیس ستونی مقدار j است (چپ تر است). حال اگر در حالت هدف اندیس ستونی i از j بیشتر باشد (راست تر باشد)، یک conflict یافت شده است.

یعنی ترتیب "هدف" با ترتیب "فعلی" مخالف است. سپس تضاد های خطی ستونی نیز به همین روال شماره گذاری میشوند.

در نهایت return ، manhattan + 2*conflicts میشود.

```
return manhattan + 2 * conflicts
```

دلیل ضرب در ۲: هر conflict حداقل ۲ حرکت اضافه تحمیل می کند. بنابراین این مقدار برای اصلاح Manhattan ضروری است

A function for printing states

تابع `print_state` مسئول نمایش حالت پازل به صورت یک ماتریس 3×3 است. این تابع با پیمایش ردیف‌های ماتریس و تبدیل عناصر عددی به رشته، هر ردیف را به صورت یک خط جداگانه چاپ می‌کند. استفاده از تابع `join` برای اتصال عناصر یک ردیف با یک فاصله، خوانایی خروجی را افزایش می‌دهد. در پایان نیز یک خط خالی چاپ می‌شود تا جداسازی حالت‌ها در خروجی کنسول به شکل مناسبی انجام گیرد.

```
def print_state(state):
    """Print a 3x3 puzzle matrix."""
    for row in state:
        print(" ".join(str(x) for x in row))
    print()
```

A function for turning state to tuple

```
def state_to_tuple(state):
    """Convert list-of-lists state into an immutable tuple form (for hashing)."""
    return tuple(tuple(row) for row in state)
```

این تابع، حالت پازل، که به صورت لیست لیست‌ها (ماتریس) است، را به یک ساختار غیرقابل تغییر (immutable) تبدیل می‌کند که:

- بتوان آن را در مجموعه‌ها قرار داد
- بتوان از آن به عنوان کلید دیکشنری استفاده کرد
- بتوان برای آن هش تولید کرد

A function for reconstructing path

این تابع مسیر کامل رسیدن از حالت شروع (start) به حالت هدف (goal) را بازسازی می‌کند. در طول اجرای الگوریتم A^* ، برای هر حالت جدید ثبت می‌کنیم که از کدام حالت قبلی به آن رسیده‌ایم. این اطلاعات در دیکشنری `came_from` ذخیره می‌شود.

```
def reconstruct_path(came_from, current):
```

این تابع دو ورودی دارد: `came_from`: دیکشنری که والد هر حالت را ذخیره می‌کند و `current`: حالت فعلی (حالت هدف)

```
path = [current]
```

در ابتدا، مسیر را با حالت هدف شروع می‌کنیم.

```
while current in came_from:
```

تا زمانی که برای حالت فعلی والد ثبت شده باشد، حلقه ادامه دارد. حالت شروع معمولاً در `came_from` وجود ندارد، بنابراین حلقه آنجا متوقف می‌شود.

```
current = came_from[current]
```

رفتن به حالت والد: در این خط، حالت قبلی (`parent`) بازیابی می‌شود.

```
path.append(current)
```

هر حالت والد به لیست مسیر اضافه می‌شود.

پس لیست به این شکل ساخته می‌شود: `[Goal, State3, State2, State1, Start]`

```
path.reverse()
```

چون مسیر را از هدف به شروع ساخته‌ایم، باید آن را برعکس کنیم تا ترتیب درست به دست آید

A function for detecting move and swap

این تابع یکی از بهترین ابزارها برای توضیح مسیر حل پازل برای انسان است، زیرا جهت حرکت و کاشی جابه‌جا شده را مشخص می‌کند.

این تابع دو حالت متوالی را می‌گیرد و مشخص می‌کند:

- صفر (خانه خالی) در چه جهتی حرکت کرده (up, down, left, right)
- با کدام کاشی جابه‌جا شده است

این اطلاعات برای تولید توضیحات مرحله به مرحله، چاپ مسیر حل و ساخت گزارش بسیار مهم است.

```
for i in range(3):
    for j in range(3):
        if prev_state[i][j] == 0:
            p0 = (i, j)
        if next_state[i][j] == 0:
            n0 = (i, j)
```

در ابتدا، در حلقه‌ی for، موقعیت قرار گیری صفر (به صورت تاپل شامل ردیف و ستون) در هر دو حالت قبلی و بعدی پیدا میشوند و در p0 و n0 ذخیره میشوند.

```
pi, pj = p0
ni, nj = n0
```

سپس موقعیت ردیف و ستون هر کدام در متغیر های مربوطه قرار می‌گیرد.

حال نوبت به شناسایی حرکت میرسد:

```
if ni == pi - 1 and nj == pj:
    direction = "up"
elif ni == pi + 1 and nj == pj:
    direction = "down"
elif ni == pi and nj == pj - 1:
    direction = "left"
elif ni == pi and nj == pj + 1:
    direction = "right"
else:
    direction = "unknown"
```

این خط کد خیلی مهم است. در حالت قبلی، با استفاده از اندیس های موقعیت صفر در حالت جدید، دقیقاً به همان کاشی که با صفر جابه‌جا شده میرسیم.

```
swapped_value = prev_state[ni][nj]
swap_info = f"swap zero with tile {swapped_value} at position ({ni},{nj})"
```

A* function

تابع a_star الگوریتم A^* را برای حل مسئله هشت پازل پیاده‌سازی می‌کند. این الگوریتم با استفاده از تابع ارزیابی $f(n)=g(n)+h(n)$ بهترین مسیر را پیدا می‌کند. این الگوریتم همواره گرهی را گسترش می‌دهد که کمترین مقدار f را دارد.

```
def a_star(start, goal, heuristic_func):
```

ورودی‌های تابع: حالت شروع پازل، حالت هدف و هیوریستیک

```
start_t = state_to_tuple(start)
goal_t = state_to_tuple(goal)
```

ابتدا حالت هدف و اولیه به تاپل تبدیل میشوند تا hashable شوند.

```
open_heap = []
g_score = {start_t: 0}
```

در ابتدا، $open_heap$ یک لیست خالی تعریف میشود.

g_score ، یک دیکشنری با کلید (حالت) و $value$ (هزینه حالت) تعریف میشود. (حالت اولیه هزینه اش صفر است)

```
f_start = heuristic_func(start, goal)
```

هیوریستیک حالت اولیه محاسبه میشود و چون هزینه مسیر در حالت اولیه صفر است، در f_start قرار میگیرد

```
heapq.heappush(open_heap, (f_start, 0, start_t))
```

با استفاده از $heappush$ ، حالت اولیه به همراه $value f$ اش و هزینه g اش (که صفر است) در $open_heap$ ، $push$ میشود.

```
generated_nodes = 0
expanded_nodes = 0
max_open_size = 1
max_closed_size = 0
```

سپس شمارنده‌های مربوط به بررسی پیچیدگی زمانی و مکانی، مقدار دهی اولیه میشوند.

```
while open_heap:
```

تا وقتی $open_heap$ خالی نشده، این حلقه ادامه میابد.

```
max_open_size = max(max_open_size, len(open_heap))
max_closed_size = max(max_closed_size, len(closed))
```

ثبت بیشترین حافظه مصرف شده برای محاسبه تقریبی Space Complexity.

```
f, g, current_t = heapq.heappop(open_heap)
```

انتخاب بهترین گره: همیشه گره با کمترین f از $heap$ خارج می‌شود.

```
if current_t in closed:
    continue
```

اگر حالت فعلی قبلاً expand شده باشد، نادیده گرفته می‌شود.

```
expanded_nodes += 1
```

افزایش شمارنده بسط داده شده‌ها

```
current_state = [list(row) for row in current_t]
```

تبدیل به لیست (چون توابع دیگر با list-of-lists کار می‌کنند).

```
closed.add(current_t)
```

اضافه کردن گره به CLOSED

```
if current_t == goal_t:
    final_path = reconstruct_path(came_from, current_t)
```

اگر به هدف رسیدیم:

۱. مسیر بازسازی می‌شود
۲. حرکت‌ها استخراج می‌شوند
۳. گزارش نهایی تولید می‌شود

```
direction, swap_info = detect_move_and_swap(prev_s, next_s)
```

استخراج حرکت‌ها برای هر دو state متوالی: جهت حرکت صفر و کاشی جابه‌جا شده

```
• swaps = len(final_path) - 1
```

محاسبه تعداد حرکات

```
space_complexity = max_open_size + max_closed_size
```

محاسبه فضای مصرف شده

```
children = generate_children(current_state)
```

اگر حالت هدف پیدا نشده بود: تولید فرزندان (همه حالت‌های قابل دسترس با یک حرکت).

```
for child in children:
```

```
if child_t in closed:  
    continue
```

اگر قبلاً بررسی شده باشد، حذف می‌شود.

```
tentative_g = g + 1
```

محاسبه g جدید

```
child_h = heuristic_func(child, goal)  
child_f = tentative_g + child_h
```

محاسبه هیوریستیک و f جدید

```
better = (  
    child_t not in g_score or tentative_g < g_score[child_t]  
)
```

این خط قرار است تصمیم بگیرد که: آیا مسیر جدیدی که به این فرزند رسیده‌ایم بهتر از مسیرهایی است که قبلاً برای همین حالت پیدا شده بود؟

خروجی آن (True/False) تعیین می‌کند که:

- اگر True: این فرزند باید وارد OPEN شود
- اگر False: بهتر است نادیده گرفته شود چون مسیر قبلی بهتر بوده

آیا این حالت را قبلاً دیده‌ایم: `child_t not in g_score`

اگر نه یعنی اولین بار است این حالت را می‌بینیم و بنابراین باید آن را به OPEN اضافه کنیم یا اگر قبلاً دیده‌ایم، آیا مسیر جدید کوتاه‌تر است؟

`tentative_g < g_score[child_t]`

اگر مسیر جدید (`tentative_g`) کوتاه‌تر از مسیر قدیمی باشد، باید مسیر بهتر را جایگزین کنیم.

```
g_score[child_t] = tentative_g  
came_from[child_t] = current_t
```

به‌روزرسانی اطلاعات مسیر

```
heapq.heappush(open_heap, (child_f, tentative_g, child_t))
```

اضافه کردن به heap

در نهایت، الگوریتم دوباره به ابتدای حلقه ی while برمیگردد...

Report and Call

===== FINAL RESULT WITH HEURISTIC manhattan_distance =====

Status: Path found successfully.

Total swaps: 26

Generated nodes: 7025

Expanded nodes: 2628

Space complexity: 4048

Move sequence:

['left', 'up', 'up', 'right', 'down', 'down', 'right', 'up', 'up', 'left', 'down', 'down', 'left', 'up', 'right', 'right', 'down', 'left', 'up', 'right', 'up', 'left', 'left', 'down', 'down', 'right']

Swap details:

swap zero with tile 3 at position (2,0)

swap zero with tile 4 at position (1,0)

swap zero with tile 7 at position (0,0)

swap zero with tile 2 at position (0,1)

swap zero with tile 5 at position (1,1)

swap zero with tile 3 at position (2,1)

swap zero with tile 1 at position (2,2)

swap zero with tile 6 at position (1,2)

swap zero with tile 8 at position (0,2)

swap zero with tile 5 at position (0,1)

swap zero with tile 3 at position (1,1)

swap zero with tile 1 at position (2,1)

swap zero with tile 4 at position (2,0)

swap zero with tile 7 at position (1,0)

...

swap zero with tile 1 at position (1,0)

swap zero with tile 7 at position (2,0)

swap zero with tile 8 at position (2,1)

===== FINAL RESULT WITH HEURISTIC manhattan_conflict =====

Status: Path found successfully.

Total swaps: 26

Generated nodes: 3768

Expanded nodes: 1440

Space complexity: 2192

Move sequence:

['left', 'up', 'up', 'right', 'down', 'down', 'right', 'up', 'up', 'left', 'down', 'down', 'left', 'up', 'right', 'right', 'down', 'left', 'up', 'right', 'up', 'left', 'left', 'down', 'down', 'right']

Swap details:

swap zero with tile 3 at position (2,0)

swap zero with tile 4 at position (1,0)

swap zero with tile 7 at position (0,0)

swap zero with tile 2 at position (0,1)

swap zero with tile 5 at position (1,1)

swap zero with tile 3 at position (2,1)

swap zero with tile 1 at position (2,2)

swap zero with tile 6 at position (1,2)

swap zero with tile 8 at position (0,2)

swap zero with tile 5 at position (0,1)

swap zero with tile 3 at position (1,1)

swap zero with tile 1 at position (2,1)

swap zero with tile 4 at position (2,0)

swap zero with tile 7 at position (1,0)

...

swap zero with tile 1 at position (1,0)

swap zero with tile 7 at position (2,0)

swap zero with tile 8 at position (2,1)

Analysis and Conclusion

در هر دو اجرا، الگوریتم A^* دقیقاً همان مسیر بهینه را با ۲۶ جابه‌جایی پیدا کرده است، اما تفاوت چشمگیر در کارایی کاملاً نشان می‌دهد که انتخاب heuristic چه تأثیری روی عملکرد دارد .

با اینکه Manhattan و Manhattan+Conflict هر دو جواب یکسان می‌دهند، heuristic دوم به مراتب دقیق‌تر است و باعث شده A^* تعداد بسیار کمتری گره بررسی کند:

-تعداد گره‌های تولیدشده تقریباً نصف شد: از ۷۰۲۵ به ۳۷۶۸ رسید.

-تعداد گره‌های گسترش‌یافته تقریباً نصف شد: از ۲۶۲۸ به ۱۴۴۰ رسید.

-پیچیدگی فضایی نیز حدود ۴۵٪ کاهش یافت

این کاهش‌ها نشان می‌دهد که Manhattan+Linear Conflict تخمین واقع‌بینانه‌تری از فاصله تا هدف ارائه می‌دهد، بنابراین A^* کمتر وارد مسیرهای اشتباه می‌شود و زودتر به گره‌های امیدبخش می‌رسد .

نتیجه: هر دو heuristic مسیر بهینه را پیدا می‌کنند، اما Manhattan+Conflict به‌طور قابل توجهی کارآمدتر است و با گسترش کمتر، حافظه و زمان کمتری مصرف می‌کند.

Answer to an important question about time complexity

آیا استفاده از تعداد گره های اضافه شونده به Q معیار مناسبی برای مقایسه پیچیدگی زمانی دو قسمت ۵ و ۶ است؟ توضیح دهید.

به طور کلی خیر. تعداد گره هایی که به صف Frontier (یا Q) افزوده می شوند تنها یکی از شاخص های جزئی برای ارزیابی عملکرد الگوریتم است، اما معیار دقیقی برای مقایسه ی پیچیدگی زمانی به شمار نمی آید. دلایل این موضوع عبارت اند از:

- (۱) صف Frontier تنها بخشی از عملیات زمانی را منعکس می کند
پیچیدگی زمانی الگوریتم های جست و جو به تعداد کل گره های تولید شده (generated) و بسط داده شده (expanded) وابسته است، بنابراین، اندازه Q فقط بخشی از بار محاسباتی را نمایش می دهد.
 - (۲) گره های اضافه شده به Q الزاماً مساوی با تعداد گسترش ها نیستند دو الگوریتم ممکن است تعداد مشابهی گره به Q اضافه کنند، اما: یکی تعداد زیادی از گره ها را گسترش دهد، دیگری تقریباً هیچ یک از آن ها را گسترش ندهد.
اما پیچیدگی زمانی واقعی بر اساس تعداد گسترش ها (expansions) و عملیات روی successor ها تعیین می شود، نه صرفاً enqueue کردن.
 - (۳) برخی الگوریتم ها Frontier بزرگی ایجاد می کنند ولی گسترش کمی دارند. برای مثال:
Greedy Best-First Search ممکن است هزاران گره را در Q نگه دارد بدون آنکه واقعاً آنها را گسترش دهد، چون همواره سراغ بهترین گره می رود. در مقابل Depth-First Search ممکن است Frontier بسیار کوچکی داشته باشی اما تعداد گسترش ها بسیار زیاد باشد.
بنابراین اندازه Q معیار مستقیمی از زمان نیست.
 - (۴) هزینه های واقعی زمانی روی عملیات دیگر صرف می شود.
پیچیدگی زمانی جست و جو به عواملی غیر از enqueue وابسته است:
 - تعداد successor های تولید شده
 - تعداد مقایسه های انجام شده در ساختار داده (به خصوص در A^* و UCS)
 - هزینه محاسبه heuristic
 - بازدیدهای تکراری و چک کردن membership در مجموعه بسته (Closed set)
 - عملیات مرتب سازی یا heap operations
- بنابراین، تعداد گره های افزوده شده به صف Frontier می تواند شاخص کمکی باشد، اما معیار اصلی و مناسب برای سنجش پیچیدگی زمانی، تعداد گره های گسترش یافته (expanded) است. به طور استاندارد معمولاً:

Time Complexity $\approx$ تعداد گره های expanded

space Complexity $\approx$ بیشترین اندازه Frontier یا مجموع یا ماکزیموم Frontier+Closed